

Frontend&Backend

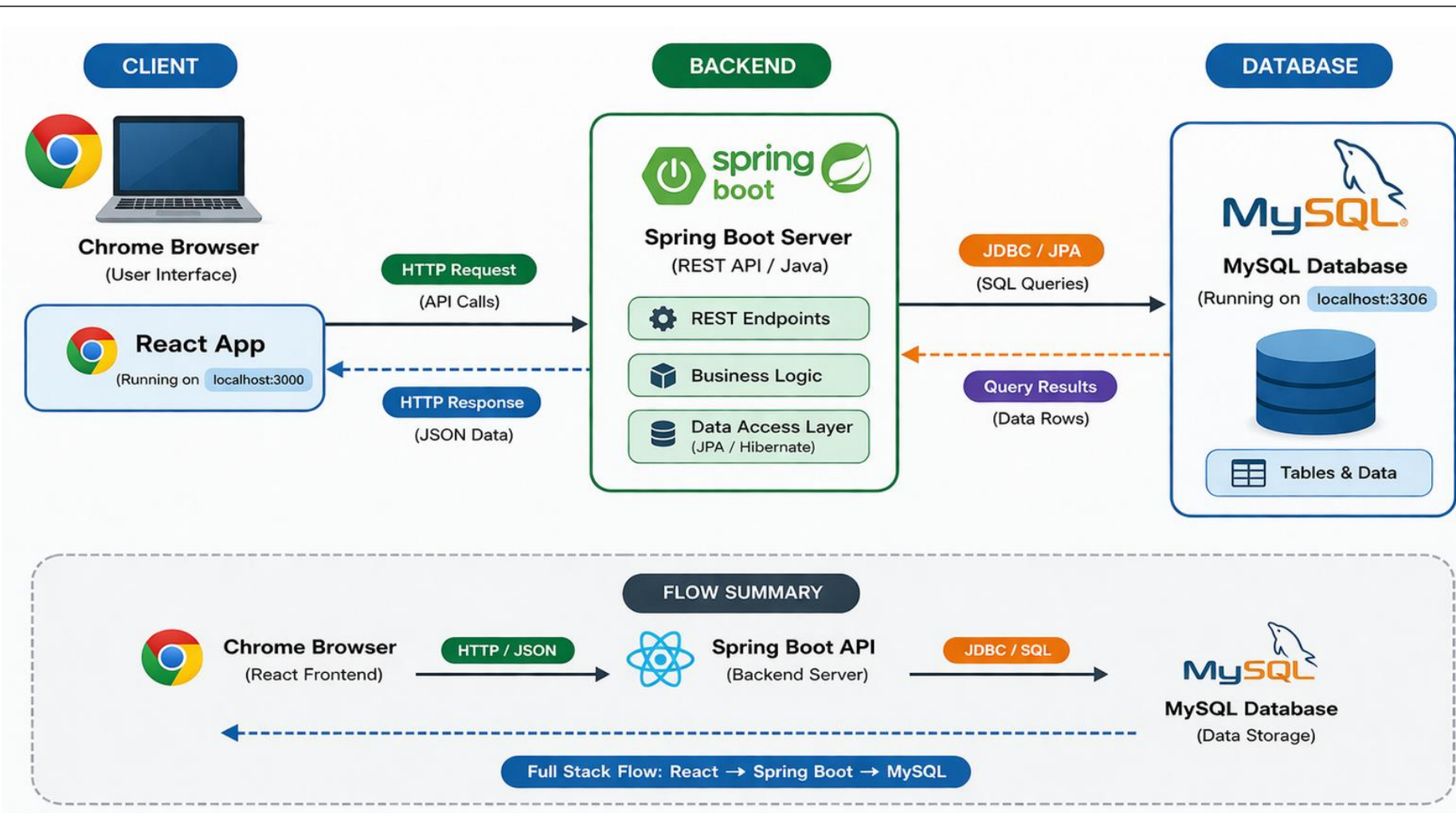
Frontend&Backend



개괄적 흐름

데이터 흐름

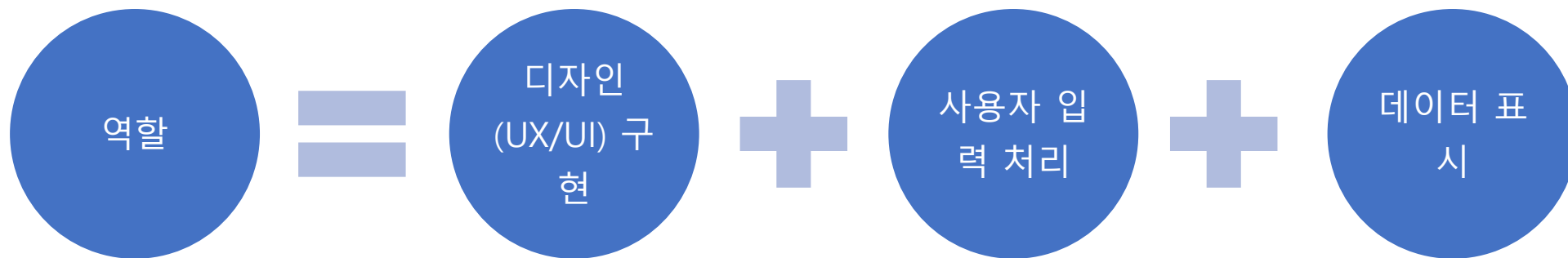
클라이언트 - 프론트엔드 - 백엔드 - 데이터베이스에 대한 개략적인 흐름입니다.



프론트엔드 개요

프론트엔드는 사용자와 직접 상호 작용하는 웹 애플리케이션의 화면 부분을 개발하는 영역입니다.

프론트엔드의 역할



핵심 목표

- 직관적인 인터페이스 제공
- 반응형 레이아웃
- 성능 최적화

☰ 프론트엔드 웹 개발 기술

☰ 기본 웹 기술 학습

프론트엔드를 하기 위한 기본 웹 개발 기술 목록은 다음과 같습니다.

- HTML 기본 구조와 시맨틱 태그
- CSS 선택자, 박스 모델, Flex/Grid 레이아웃
- JavaScript 기본 문법(변수, 함수, 조건문, 반복문)
- DOM 조작과 이벤트 처리



≡ 웹 개발 기술의 기본

프론트엔드는 사용자가 직접 눈으로 보고, 클릭하고, 입력하는 화면(UI) 부분을 담당합니다.

(1) HTML (HyperText Markup Language)

- 역할 : 웹 페이지 문서의 구조(뼈대)를 정의
- 예시 : 제목, 단락, 이미지, 표, 링크, 입력창, 버튼 등

예시 코드

```
<h1>나의 첫 번째 웹페이지</h1>  
<p>안녕하세요! 이것은 HTML 예제입니다.</p>  

```

≡ 웹 개발 기술의 기본

프론트엔드는 사용자가 직접 눈으로 보고, 클릭하고, 입력하는 화면(UI) 부분을 담당합니다.

(2) CSS (Cascading Style Sheets)

- 역할 : HTML로 만든 구조를 보기 좋게 디자인
- 예시 : 글자 색, 배경색, 버튼 모양, 레이아웃 배치

예시 코드

```
h1 {  
  color: blue;  
  text-align: center;  
}  
p {  
  font-size: 18px;  
}
```

☰ 웹 개발 기술의 기본

프론트엔드는 사용자가 직접 눈으로 보고, 클릭하고, 입력하는 화면(UI) 부분을 담당합니다.

(3) JavaScript (JS)

- 역할 : 웹 페이지에 동작(인터랙션)을 추가
- 예시 : 버튼 클릭 시 메시지 출력, 메뉴 열고 닫기, 입력값 확인, 애니메이션 등

예시 코드

```
document.getElementById("btn").addEventListener("click", function() {  
    alert("버튼이 눌렸습니다!");  
});
```

웹 개발 기술의 기본

프론트엔드는 사용자가 직접 눈으로 보고, 클릭하고, 입력하는 화면(UI) 부분을 담당합니다.

(4) XML

- 역할 : 데이터를 저장하고 전달하기 위한 구조화된 문서 형식
- 예시 : 설정 파일(config), 데이터 교환(API 응답), 문서 구조 정의 (태그 기반 데이터 표현)

예시 코드

```
<?xml version="1.0" encoding="UTF-8"?>
<message>
  <button id="btn">Click Me</button>
  <event type="click">
    <text>버튼이 눌렸습니다!</text>
  </event>
</message>
```


웹 개발 기술의 기본

프론트엔드는 사용자가 직접 눈으로 보고, 클릭하고, 입력하는 화면(UI) 부분을 담당합니다.

(5) JSON

- 역할 : 데이터를 저장하고 전달하기 위한 경량 데이터 형식
- 예시 : API 응답 데이터, 설정 파일(config), 서버-클라이언트 데이터 교환

예시 코드

```
{
  "message": {
    "button": {
      "id": "btn",
      "text": "Click Me"
    },
    "event": {
      "type": "click",
      "text": "버튼이 눌렸습니다!"
    }
  }
}
```

프론트엔드 웹 개발 기술

프론트엔드 핵심

초급 프로젝트에서는 Vanilla JS, 이후에는 React, Vue, Angular 같은 프레임워크로 확장하면 좋습니다.



React



Vue



Angular JS

프레임워크 비교

구분	React	Vue.ks	Angular
개발사	Meta(Facebook)	커뮤니티 (Evan You 주도)	Google
성격	라이브러리	경량 프레임워크	대규모 프레임워크
언어	JavaScript + JSX	JavaScript + 템플릿	TypeScript
구조	컴포넌트 기반	컴포넌트 기반	MVC 아키텍처
데이터 바인딩	단방향 중심	양방향 지원	양방향 지원
난이도	중간	쉬움	어려움
활용 사례	대규모 웹앱, 페이스북, 넷플릭스	빠른 개발, 알리바바	엔터프라이즈, 구글 서비스

☰ 프론트엔드 웹 개발 기술

☰ 프론트엔드 핵심

초급 프로젝트에서는 Vanilla JS, 이후에는 React, Vue, Angular 같은 프레임워크로 확장하면 좋습니다.

- ES6+ 문법(let/const, 화살표 함수, 템플릿 리터럴, 모듈 등)
- 비동기 처리(Promise, async/await, fetch API)
- 브라우저 동작 원리(렌더링, 이벤트 루프)

☰ 프레임워크/라이브러리 학습

프레임워크/라이브러리 학습

- React 기본(컴포넌트, props, state, 이벤트)
- React Router, 상태 관리(useContext, Redux 등)
- UI 프레임워크 활용(Bootstrap, Tailwind, Material UI 등)

프론트엔드 웹 개발 기술

프론트엔드 개발 환경

프론트엔드 개발 환경 및 도구들에 대한 숙지가 있어야 합니다.

프론트엔드 도구 및 개발 환경

항목	설명
코드 에디터	VS Code
버전 관리	Git/GitHub을 통한 버전 관리
패키지 매니저	Node.js(npm), yarn
개발자 도구	Chrome DevTools 활용



Node.JS



Git



GitHub

백엔드 개요

사용자의 요청을 처리하고 데이터와 비즈니스 로직을 관리하는 서버 측 개발 영역을 의미합니다.

백엔드의 역할



핵심 목표

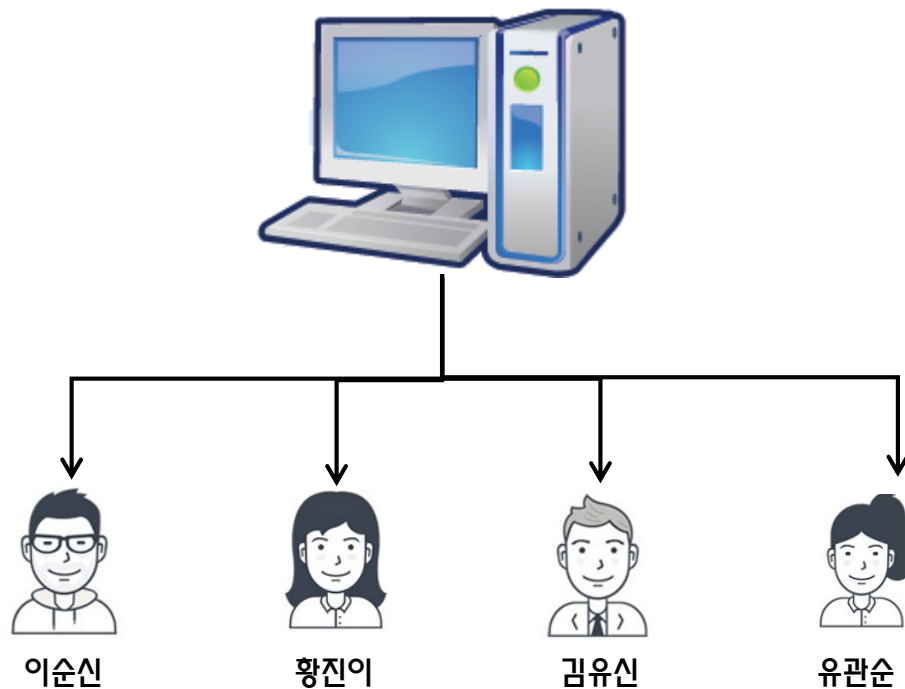
- 안정성
- 확장성
- 보안성

서버와 클라이언트 이해

서버와 클라이언트에 대한 기본 이해가 필요합니다.

웹 페이지는 기본적으로 Client와 Server 환경(CS 환경)으로 동작합니다.

이러한 구조를 클라이언트/서버 구조라고 하며, 기본적으로 TCP/IP라는 프로토콜을 사용하여 네트워크 프로그램입니다.



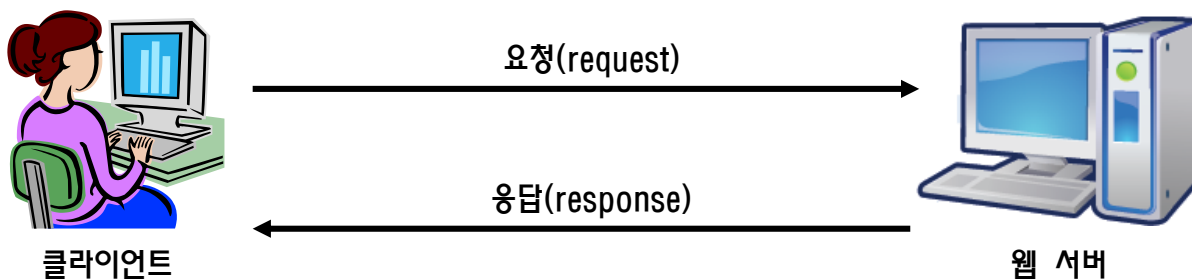
클라이언트/서버 구조

☰ 서버와 클라이언트 이해

서버와 클라이언트에 대한 기본 이해가 필요합니다.

클라이언트는 특정 페이지를 웹 서버에게 요청합니다.

요청을 받은 해당 서버는 이에 대한 적절한 응답 데이터를 만들어서 클라이언터에게 되돌려 줍니다.



☰ 백엔드 개발 기술

☰ 백엔드 언어 학습

백엔드와 관련된 프로그래밍 언어도 적절히 선택하여야 합니다.

- Java : Spring Boot 기본 구조(Controller, Service, Repository)
- Python : Flask/Django 기초
- Node.js : Express.js 기본 라우팅 및 미들웨어



Java



SpringBoot



Python



Node.JS

☰ 데이터베이스 학습

데이터베이스에 대한 기본적인 학습이 되어 있어야 합니다.

DB 종류

- 관계형 (MySQL, PostgreSQL)
- 비관계형 (MongoDB, Redis)

SQL 기초

- 기본 SQL(SELECT, JOIN, GROUP BY)
- CRUD : Create, Read, Update, Delete

ORM 활용

- JPA/Hibernate/MyBatis, Sequelize 등



Oracle



MySql



MyBatis

≡ 백엔드 개발 환경

백엔드 개발 환경 및 도구들에 대한 숙지가 있어야 합니다.

백엔드 도구 및 개발 환경

항목	설명
IDE/에디터	IntelliJ, VS Code, PyCharm
버전 관리	Git/GitHub을 통한 버전 관리
빌드/실행 도구	Maven, Gradle, npm
배포 플랫폼	AWS, Heroku, Vercel

☰ 인증 및 보안

다음과 같은 인증 및 보안에 대한 이해가 필요합니다.

- 세션/쿠키, JWT 이해
- 회원 가입/로그인 기능 구현
- 비밀번호 암호화(bcrypt 등)

☰ 배포 및 운영 기초

배포 및 운영 기초

- 로컬 → 클라우드 환경으로 배포(AWS, OCI, Heroku 등)
- Docker를 활용한 환경 설정
- 기본 로그/에러 처리
- CI/CD